GLOBALPAY

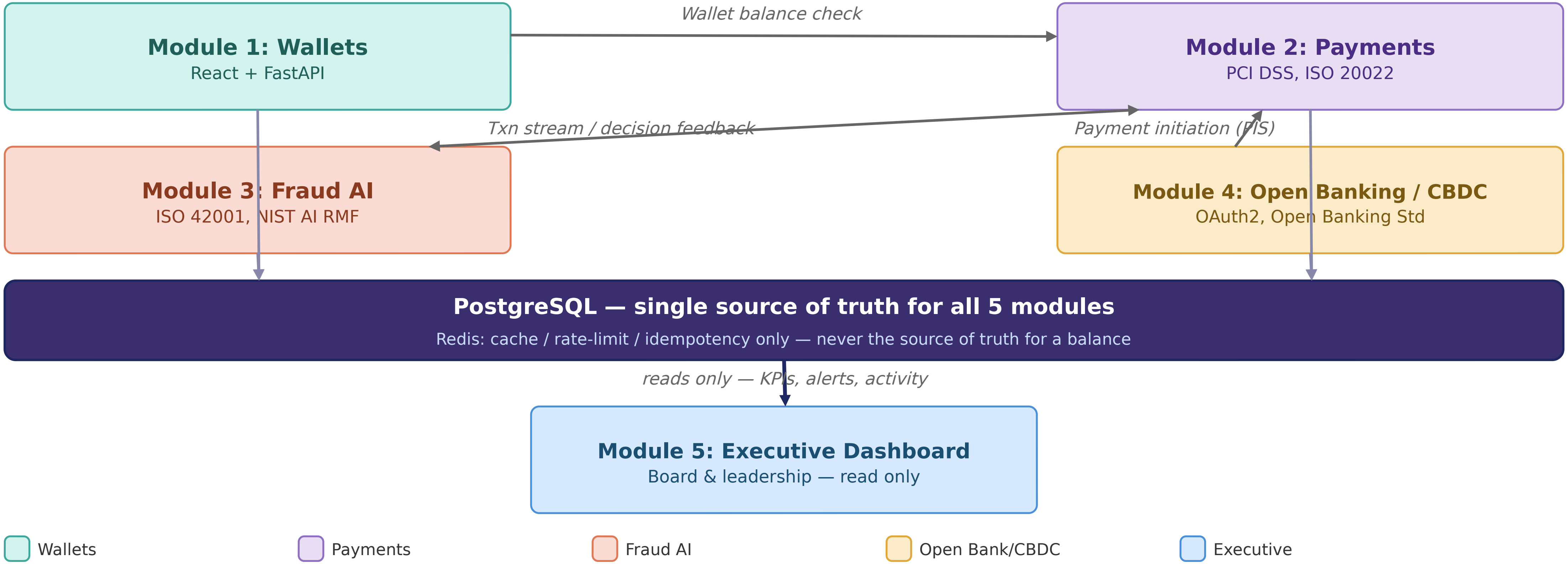
Mandatory Architecture & Engineering Deliverables

All 17 mandatory diagrams, numbered exactly as listed in the exam brief

#	Diagram	Also in Stage 1 deck?
1	Enterprise Financial Technology Architecture	Yes
2	High-Level Solution Architecture	No - here only
3	Detailed System Architecture	No - here only (composite index)
4	Digital Wallet Architecture	Yes (Module 1)
5	Payment Processing Architecture	Yes (Module 2)
6	Open Banking API Architecture	Yes (Module 4)
7	AI Fraud Detection Architecture	Yes (Module 3)
8	CBDC Simulation Architecture	No - here only
9	Enterprise Network Architecture	Yes
10	Network Flow Diagram	Yes (combined in Module 11)
11	Data Flow Diagram (Level 0)	Yes (combined)
12	Data Flow Diagram (Level 1)	Yes (combined)
13	Digital Payment Workflow	Yes (combined in Module 2)
14	Fraud Detection Workflow	Yes (combined in Module 3)
15	Open Banking API Workflow	Yes (combined in Module 4)
16	Database ERD	Yes
17	Sequence Diagram	No - here only

Enterprise Financial Technology Architecture

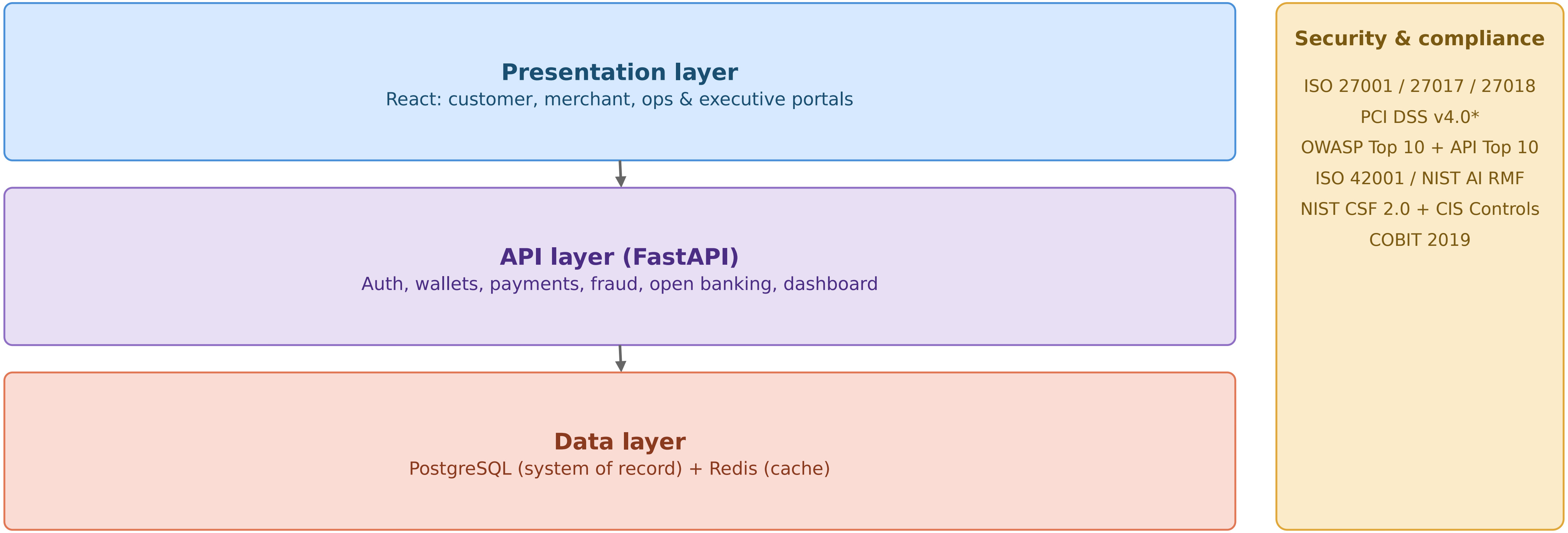
How the five modules connect



Modules 1–4 write to PostgreSQL; Module 5 reads only; only the named service calls above cross module boundaries

High-Level Solution Architecture

Layered platform view



Security and compliance controls apply across the presentation, API and data layers

G

Detailed System Architecture

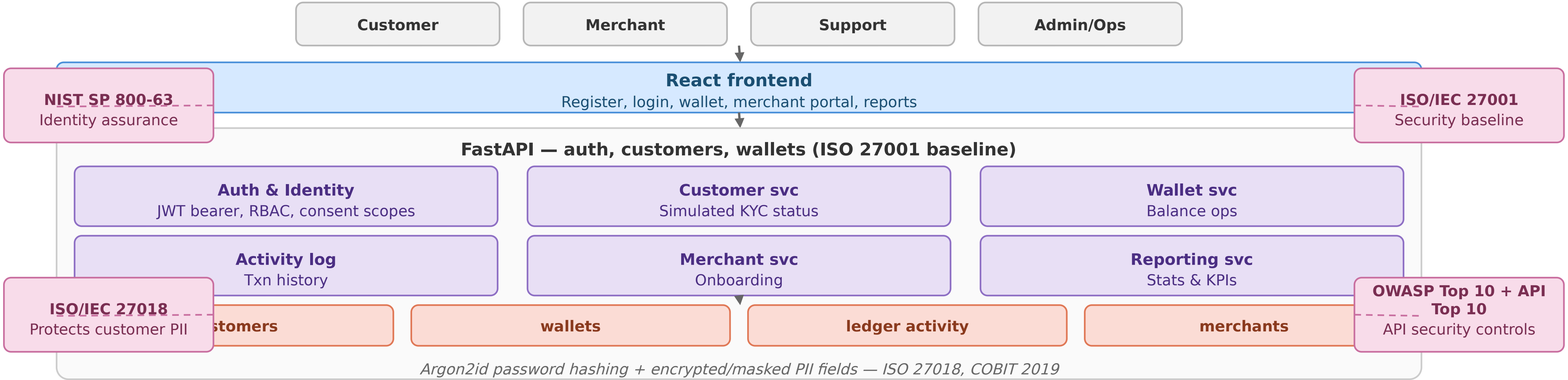
Five implementation modules and their primary responsibilities

Module	Responsibilities
Module 1 Wallets	React UI -> FastAPI (auth, customers, wallets, merchants) -> PostgreSQL
Module 2 Payments	Validation, transaction engine, double-entry ledger -> ledger tables
Module 3 Fraud AI	Feature engineering -> Logistic Regression -> risk scoring -> human review
Module 4 Open Banking/CBDC	AIS/PIS APIs, CBDC API, activity monitoring -> separate CBDC ledger
Module 5 Executive Dashboard	Read-only aggregator across Modules 1-4 -> executive reports

Each module follows the shared React, FastAPI and PostgreSQL architecture with common security, audit and reporting controls

Digital Wallet Architecture

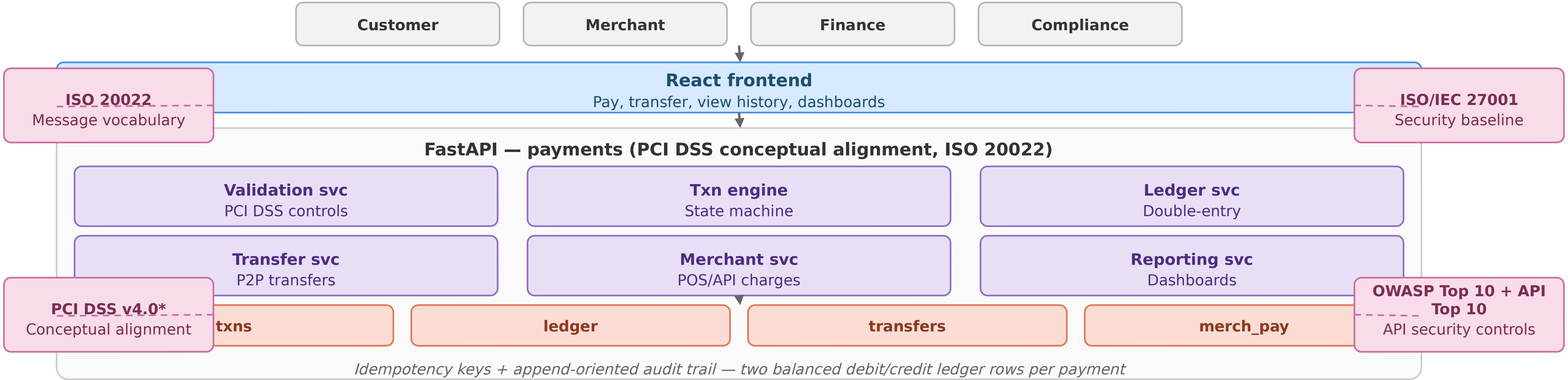
Registration, wallet creation, balance operations, merchant accounts



Wallet and identity operations write directly to PostgreSQL

Payment Processing Architecture

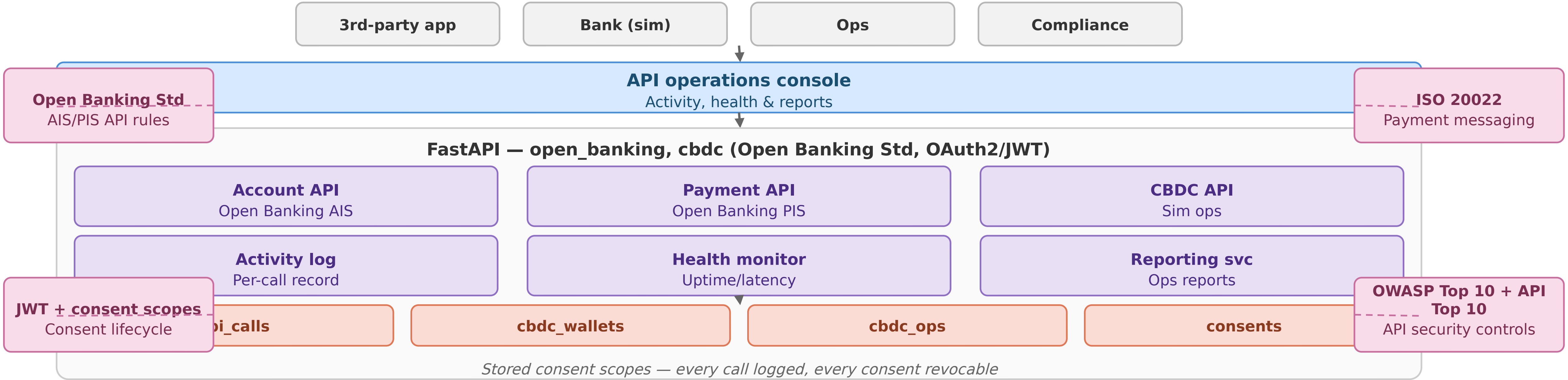
Validation, double-entry ledger, transfers, merchant charges



Direct link to Module 3 for fraud scoring — the only true module-to-module call besides Module 4's PIS

Open Banking API Architecture

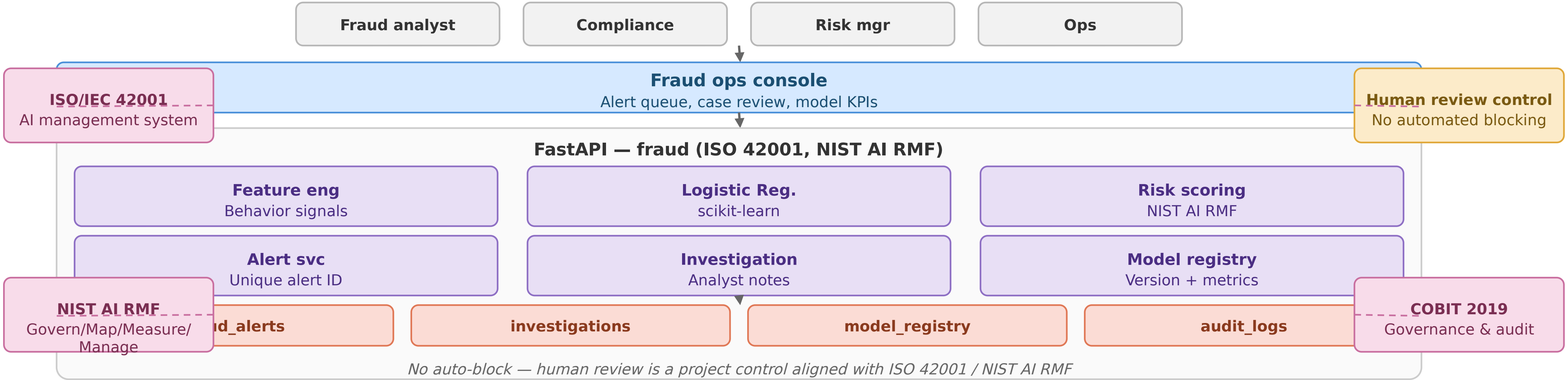
Account info & payment initiation APIs, CBDC wallet ops



PIS reuses Module 2 transfer logic; CBDC wallet and operation records remain separate from commercial wallets

AI Fraud Detection Architecture

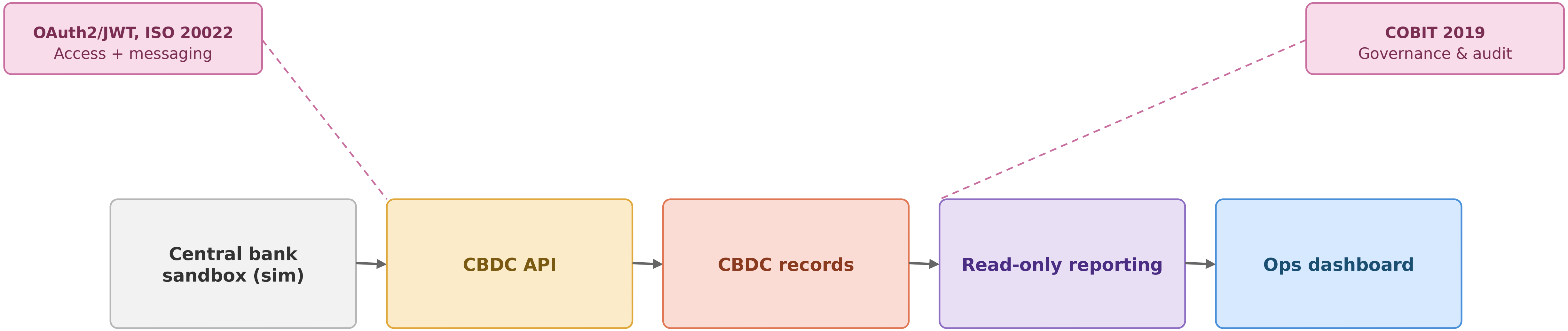
Feature engineering, classification, mandatory human review



Surfaces to fraud/risk/compliance teams only — never shown to customers

CBDC Simulation Architecture

Kept separate from the commercial wallet ledger

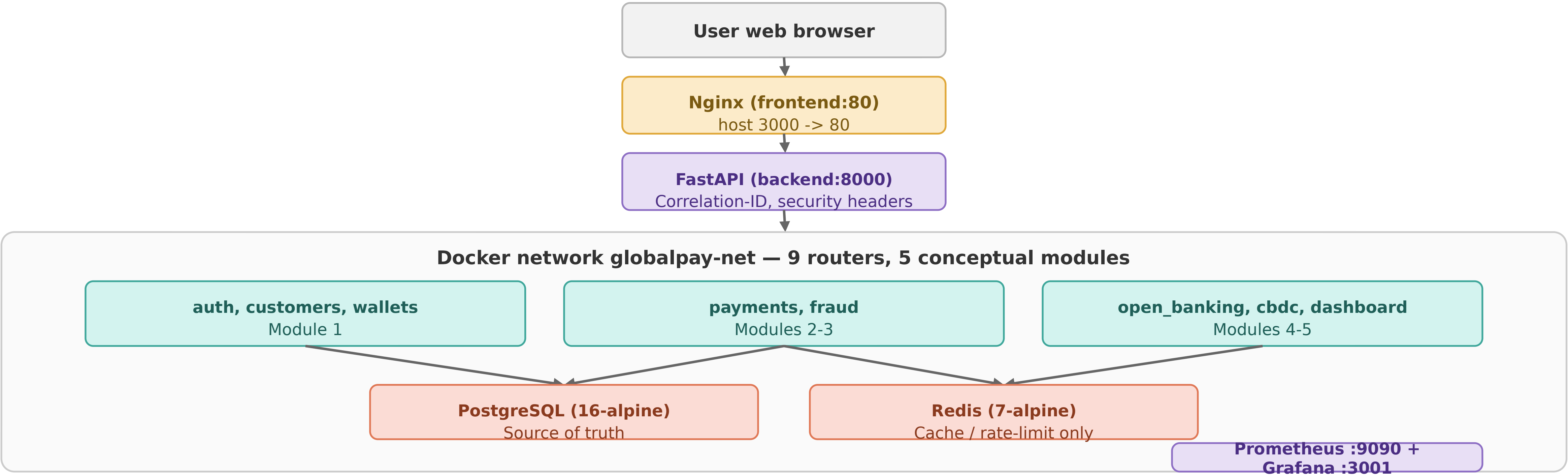


Reporting reads cbdc_wallets and cbdc_operations; these records remain separate from commercial wallet data

Central-bank money is never mixed with commercial wallet balances — this table separation is the audit boundary

Enterprise Network Architecture

compose.yaml — 6 containers on one Docker network (globalpay-net)



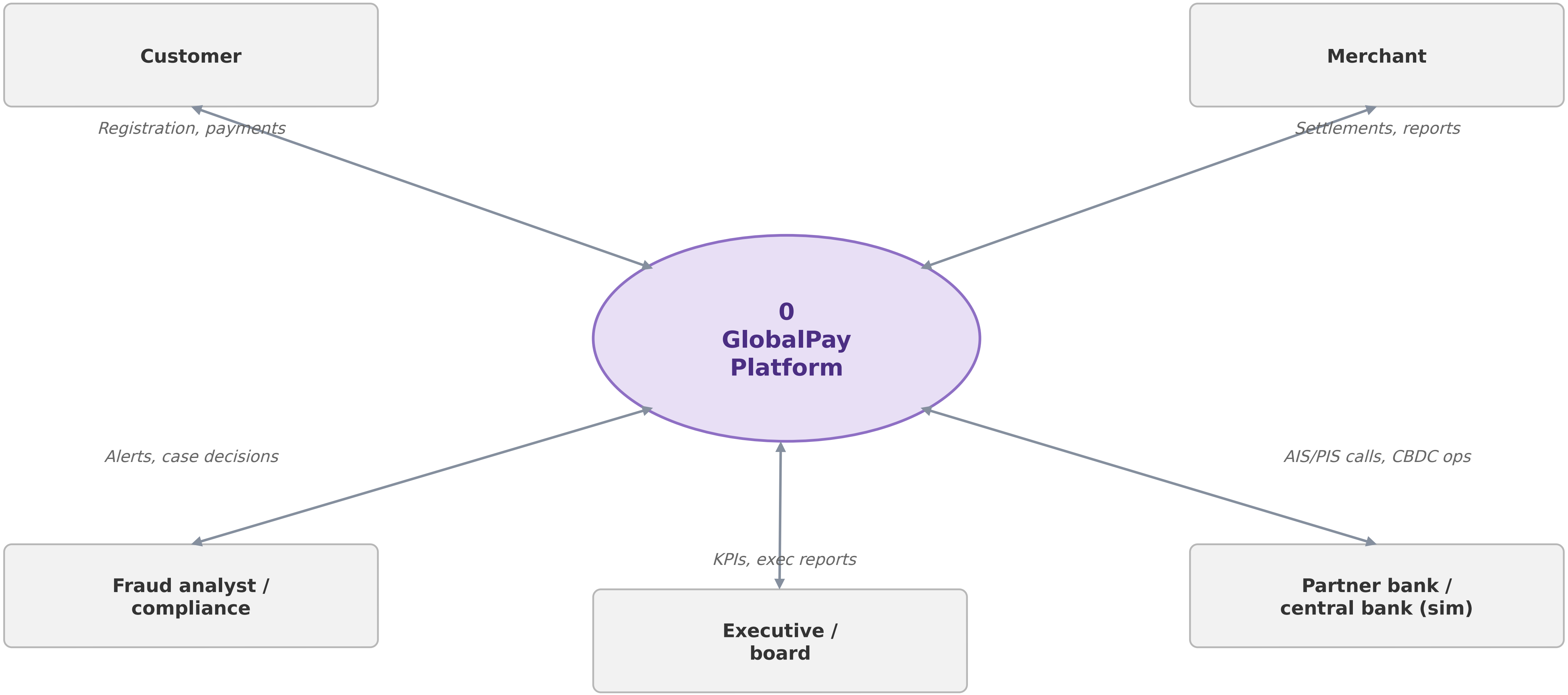
Redis never holds the authoritative wallet balance — Postgres is the single source of truth

Network Flow Diagram

How a single request traverses the network tier by tier

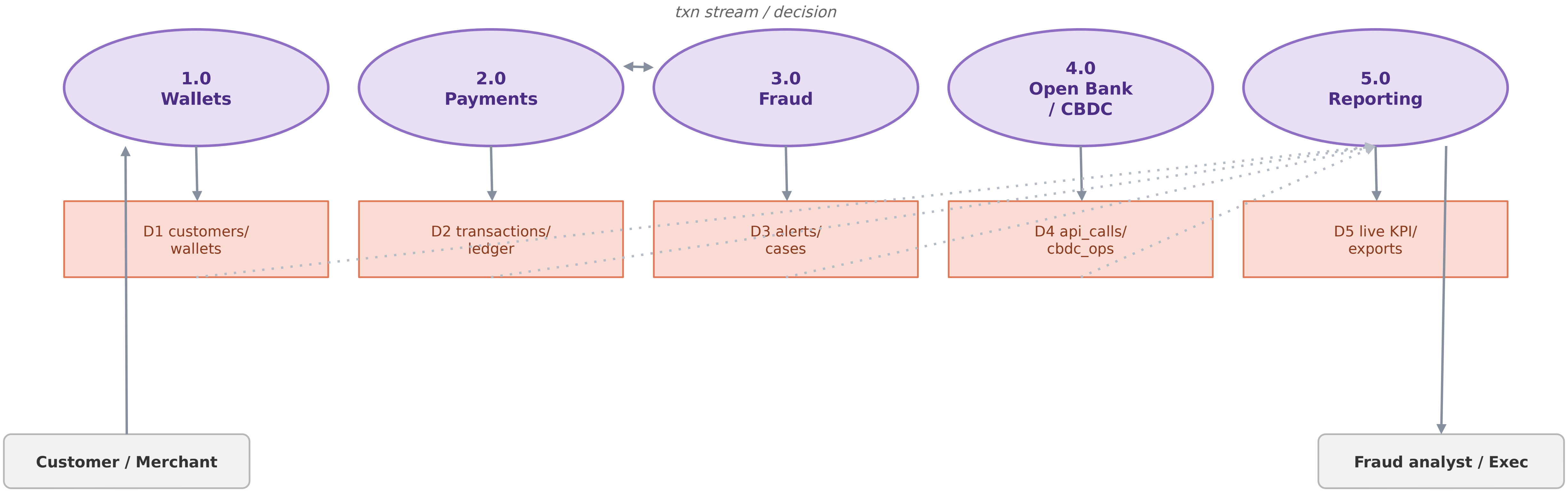
Data Flow Diagram — Level 0 (Context)

The platform as a single process, with its external entities



Data Flow Diagram — Level 1

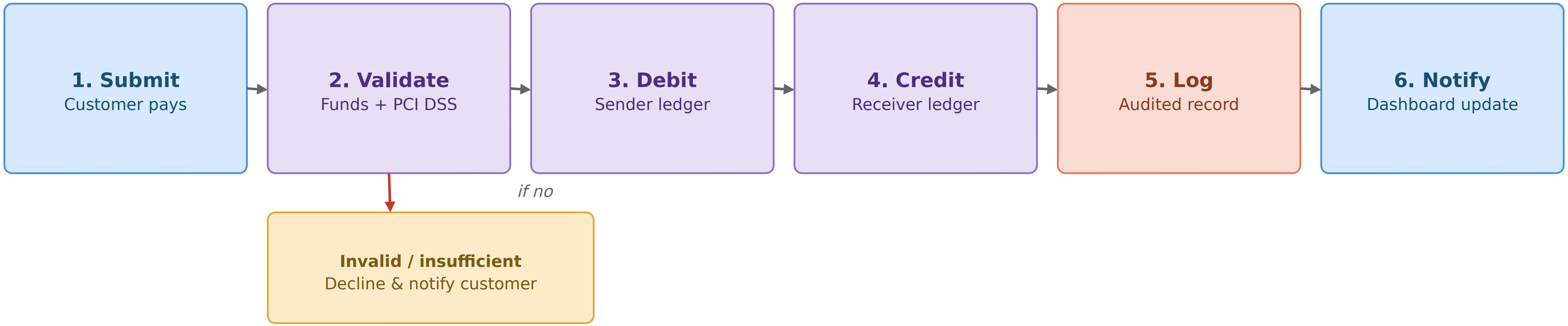
The platform decomposed into its five core processes



Dashed arrows: on-demand aggregation from data stores into 5.0 Reporting

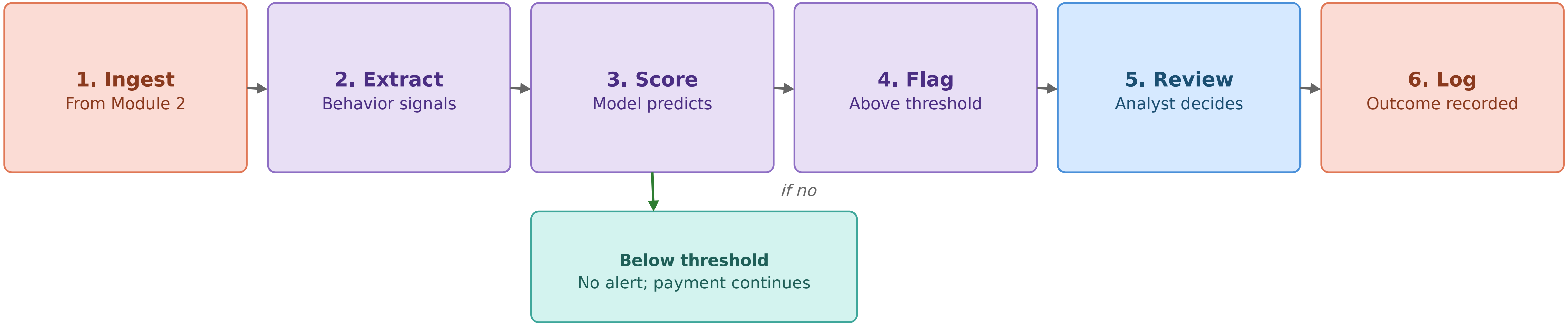
Digital Payment Workflow

Validation branch shown explicitly



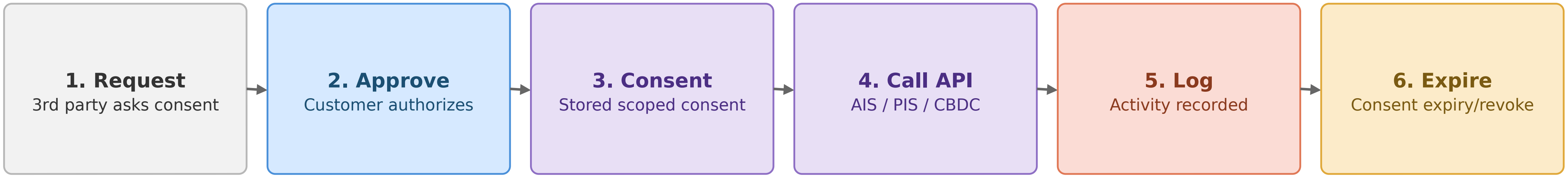
Fraud Detection Workflow

Automated scoring with mandatory human checkpoint



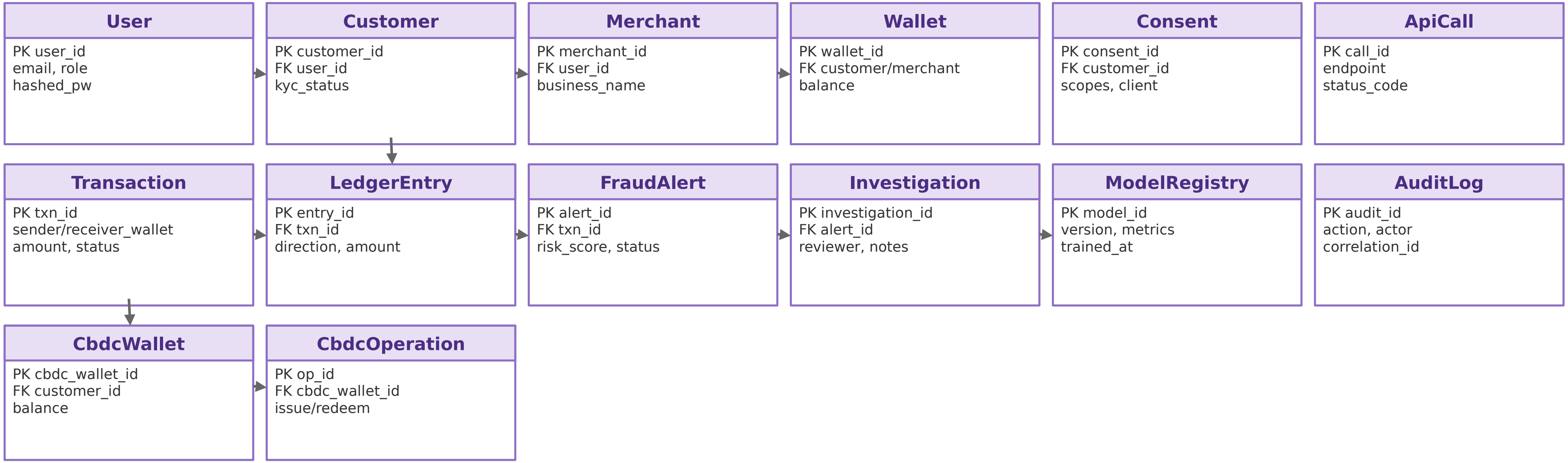
Open Banking API Workflow

Consent-first access to account and payment APIs



Database Entity Relationship Diagram (ERD)

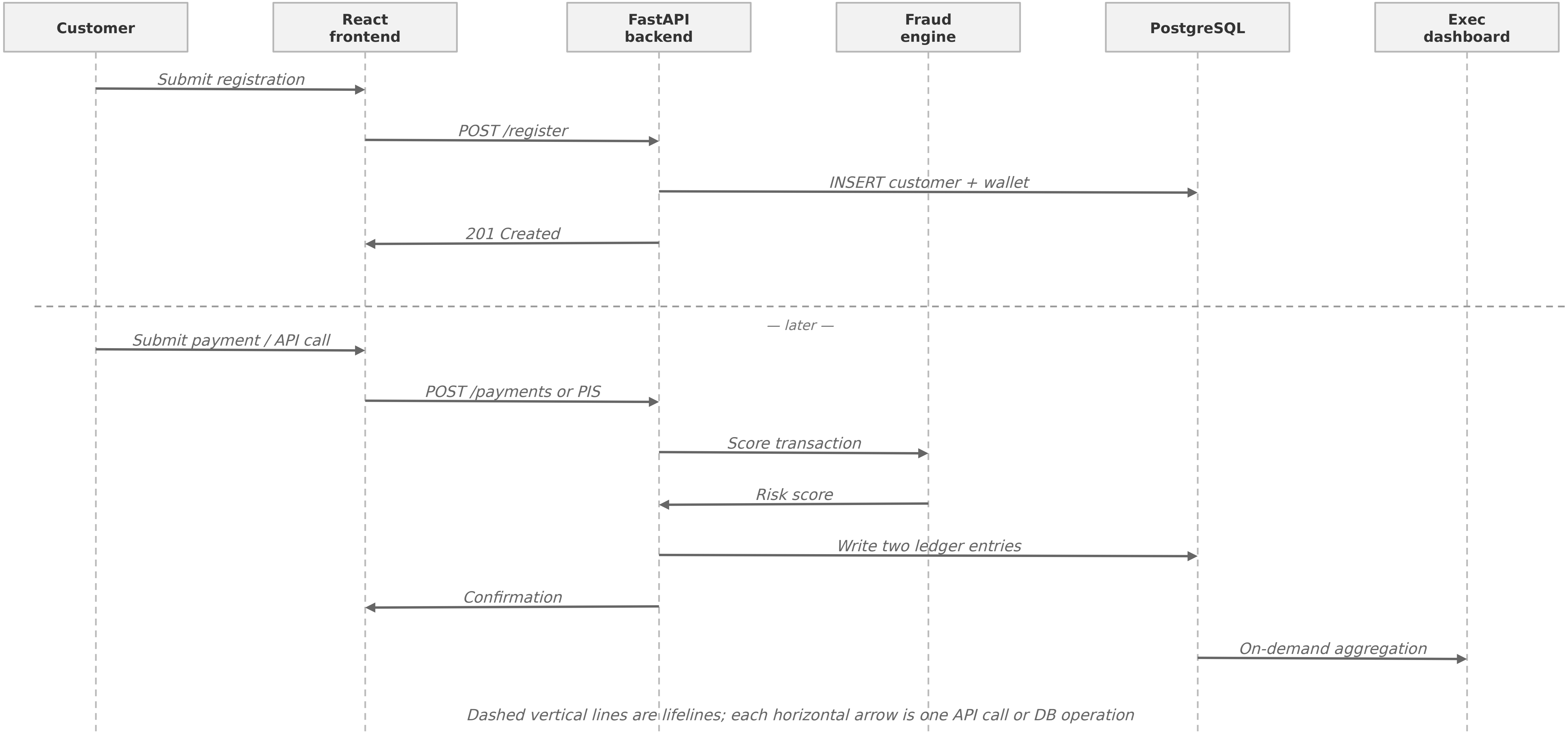
14 tables from backend/app/models/entities.py



User -> {Customer, Merchant}; Customer -> {Wallet, Consent, CbdcWallet}; Merchant -> Wallet. Transaction -> {LedgerEntry, FraudAlert -> Investigation}. ModelRegistry and AuditLog are cross-cutting.

Sequence Diagram — Registration Through Executive Reporting

End-to-end lifecycle across all five modules



Dashed vertical lines are lifelines; each horizontal arrow is one API call or DB operation